

El lenguaje ensamblador del MIPS R2000

Ing. Walter Lozano
Ing. Alejandro Rodríguez Costello

Junio 2019

1. Control del flujo de instrucciones

En assembly las instrucciones se ejecutan secuencialmente una detrás de la otra. Para ello, luego de cargar una instrucción en el procesador, se actualiza el *program counter* o PC, incrementándolo en 4 y así sucesivamente hasta que alguna instrucción cambie su valor o ocurra algún evento o excepción¹. El PC no es directamente manipulable por el programador y hay dos tipos de instrucciones que permiten cambiar su valor como consecuencia de su ejecución los saltos o *jump* y las bifurcaciones o *branch* conocidos también como saltos condicionales.

El lenguaje assembly no dispone de estructuras de control de flujo de programa como *if*, *while* o *for* que se encuentran en lenguajes de alto nivel como C, pero se puede implementar cualquier estructura de este tipo si aprendemos cómo evaluar previamente una condición, simple o compuesta. El camino que seguirá la ejecución del programa dependerá del resultado de esta evaluación.

1.1. Saltos condicionales

En MIPS se incluye dos instrucciones básicas de toma de decisiones: **beq** y **bne**, que permiten implementar estructuras de control muy sencillas. Ambas comparan el contenido de dos registros y según el resultado de esta comparación, cierta o falsa, saltan o no, a la dirección de la instrucción que referencia una etiqueta dada (dirección de memoria). La sintaxis de ambas puede resumirse:

`bxx rs, rt, etiqueta`

donde **xx** es el tipo de comparación, **eq** para el caso igual y **ne** para el caso diferente. Los registros **rs**, **rs** pueden ser cualquiera de los 32 registros del banco de registros. Para el caso de **beq** la evaluación es cierta si el contenido de ambos registros es el mismo, en cuyo caso saltará a la dirección de memoria donde la etiqueta lo indique. De lo contrario seguirá con la ejecución de la siguiente instrucción. Técnicamente esta instrucción modifica el valor del PC para que apunte a dicha dirección.

También se dispone de instrucciones de salto condicional como **bgez**, **bgtz**, **blez** y **bltz** para realizar comparaciones entre un registro y cero cuya sintaxis puede resumirse:

`bxxx rs etiqueta`

¹Más adelante abordaremos su significado

donde xxx es el tipo de comparación `rs >= 0`, `rs >0`, `rs <= 0` y `rs < 0` respectivamente.

Adicionalmente se dispone de la instrucción `slt`. Esta compara dos registros y carga un 1 ó un 0 en un tercer registro dependiendo del resultado de la comparación, cuya sintaxis es:

```
slt rd, rs, rt
```

Si el contenido de `rs` es menor que el de `rt` entonces carga un 1 en `rd`, en caso contrario carga un 0. Esta instrucción en conjunto con `beq` o `bne` permite implementar más condiciones usando pseudoinstrucciones `bgt`, `bge`, `blt`, `ble`, `sle`, `sge`, `sgt`, `seq` y `sne`².

1.2. Evaluación de expresiones

En la mayoría de los lenguajes de programación de alto nivel se dispone de la posibilidad de evaluar una condición como *true* o *false*. Si definimos a 1 como *true* y 0 como *false* entonces el siguiente código puede interpretarse como la expresión en C `char out = data1 < data2`:

```
.data
data1: .word 30
data2: .word 40
out:   .space 1
       .text

main:  lw $t0, dato1
       lw $t1, dato2
       slt $t2, $t0, $t1    # poner a 1 $t2 si t0 < t1
       sb $t2, res
```

Ejecuta el código y comprueba el valor en memoria de `out`. Cambia el código para inicializar `data1` y `data2` con valores diferentes y comprueba nuevamente `out` para entender como opera la instrucción `slt`.

Para entender la importancia de las pseudoinstrucciones veamos como implementar la expresión `char out = data1 <= data2`:

```
.data
data1: .word 30
data2: .word 40
out:   .space 1
       .text

main:  lw $t0, dato1
       lw $t1, dato2
       slt $t2, $t1, $t0
       ori $at, $0, 1
       subu $t2, $at, $t2
       sb $t2, res
```

²Para mayor referencia consulte el resumen técnico de instrucciones provisto por MIPS

Puedes comprobar prácticamente que el código hace lo que se le pide aunque su implementación es extremadamente intrincada para una persona que se inicia. Lo ideal es utilizar la pseudoinstrucción `sle` como se expresa a continuación:

```

        .data
data1:   .word 30
data2:   .word 40
out:     .space 1
        .text

main:    lw $t0, dato1
        lw $t1, dato2
        sle $t2, $t0, $t1
        sb $t2, res

```

A la evaluación de la expresión anterior se la conoce como evaluación simple, pero aplicando operadores lógicos se puede construir evaluaciones mucho más complejas como `char out = data1 < data2 && data2 > 0`:

```

        .data
data1:   .word 30
data2:   .word 40
out:     .space 1
        .text

main:    lw $t0, dato1
        lw $t1, dato2
        slt $t2, $t0, $t1    # poner a 1 $t2 si t0 < t1
        beq $t2, $0, end     # si t2 = 0 no sigo evaluando
        sgt $t3, $t1, $0     # poner a 1 $t3 si t1 > 0
        and $t2, $t2, $t3    # t2 es igual a t2 and t3
end:     sb $t2, res

```

En el código anterior se puede ver la importancia del salto condicional al evaluar. Tal como se realiza en C si la primera evaluación no es cierta no se sigue evaluando.

1.3. Bifurcaciones

Ahora que disponemos de las herramientas para evaluar expresiones podemos implementar fácilmente *if-then* ó *if-then-else*. Veamos un ejemplo sencillo para el siguiente pseudocódigo:

```

int data1 = 40;
int data2 = 30;
int res;
if (data2 != 0) {
    res = data1/data2;
} else {
    res = data1;
}
res = res + data1;

```

El siguiente código es una posible implementación práctica. En la misma se utiliza la instrucción `j label` que permite saltar a cualquier dirección apuntada por la etiqueta *label* dentro del área de texto inmediatamente. A continuación carga el código y experimenta con diferentes datos para comprobar que su ejecución guarda el valor correcto en la variable resultado `res`.

```

        .data
data1:   .word 30
data2:   .word 40
res:     .space 4
        .text

main:    lw $t0, data1
        lw $t1, data2
        beqz $t1, else
        div $t2, $t0, $t1
        j end
else:    move $t2, $t0
end:     add $t2, $t2, $t0
        sw $t2, res

```

Piensa como implementarías una condición más compleja como la que se presenta a continuación `if (data1 >= 0 && data2 > 0) res = data1/data2;` utilizando los conocimientos previos sobre expresiones. Para comprobar su funcionamiento utiliza el simulador con diferentes datos.

1.4. Iteraciones

Una vez que se ha podido implementar evaluaciones y bifurcaciones en código assembly implementar estructuras de control de flujo como *while* o *for* es relativamente trivial. A continuación se desarrolla un ejemplo de estructura de repetición para contar caracteres.

```

        .data
string:  .asciiz "Hola mundo\n"
        .align 2
length:  .space 4
        .text

main:    la $t0, string      # puntero al inicio de la cadena
        li $t1, 0           # limpiar contador
while:   lb $t2, ($t0)
        beqz $t2, end
        addi $t1, $t1, 1
        addi $t0, $t0, 1
        j while
end:     sw $t1, length

```

Es importante resaltar el porqué es necesario el contador `$t1` cuya función es muy diferente de `$t0` que es el puntero a `string`. Mientras que el primero es un número entero el segundo es una dirección de memoria. El pseudocódigo del programa anterior es muy sencillo:

```
char[] = "Hola mundo\n";
int length = 0;
while(*(string++) != 0) length++;
```

Intenta modificarlo para que no permita una longitud arbitraria sino que se detenga si la repetición ocurre más de 20 veces (el *while* utiliza una expresión compuesta).

La segunda estructura de control de flujo más conocida es *for*. En la misma se exige la presencia de tres elementos:

- Un contador que debe ser correctamente inicializado en mínimos (incremento) o máximos (decremento).
- Un método de conteo que debe incrementar o decrementar dicho contador.
- Una evaluación de la condición de salida de dicho contador.

Utilizemos como ejemplo obtener la suma de un vector de 6 enteros:

```
.data
vector: .word 6,7,8,9,10,1
sum:    .space 4
        .text

main:    li $t0,0          # limpiar acumulador sum
        li $t1,0          # limpiar contador for
        li $t2,6          # tope
loop:    la $t3,vector
        sll $t4,$t1,2      # ajusta a palabra
        add $t3,$t3,$t4    # calculo puntero
        lw $t3,($t3)
        add $t0,$t0,$t3    # acumulo
        blt $t1,$t2,loop  # testeo salida
        add $t1,$t1,1      # incremento contador for
        j loop
end:     sw $t0,sum
```

El fragmento de código se corresponde con el siguiente pseudocódigo:

```
int vector[6] = {6,7,8,9,10,1};
int sum = 0;

for (int i = 0; i < 6; i++) {
    sum = sum + vector[i];
}
```

Este es un ejemplo perfecto donde podemos observar que la implementación directa del pseudocódigo puede ser sub-óptima. En general en assembly es mejor decrementar y comparar contra 0 que incrementar y comparar contra un tope. El código anterior puede ser reescrito de manera mucho más elegante pensando en puro assembly de MIPS de la siguiente manera:

```

main:    li $t0,0           # limpiar acumulador sum
         la $t1,vector
         li $t2,6           # tope
loop:    lw $t3,($t1)
         addi $t1,$t1,4
         addi $t2,$t2,-1
         add $t0,$t0,$t3     # acumulo
         bnez $t2,loop      # testeo salida
end:     sw $t0,sum

```

1.5. Bucles anidados

En muchos algoritmos es común que un bucle se encuentre dentro de otro y que la condición de salida de un bucle interior esté regulada por otro más externo. Un ejemplo clásico es el método de ordenamiento ascendente *bubblesort* o burbuja. En este método es necesario iterar por todos los elementos conmutando de a dos en caso de no hallarse ordenados. Este paso se debe repetir nuevamente dejando de lado el último elemento del paso anterior.

```

.data
num:     6
vect:    2,23,50,1,80,22

.text
main:    la $t0, vect       # v (int *)
         lw $t1, num        # i
         addi $t2, $t1, -1  # j = i - 1
L1:      lw $t3, ($t0)
         lw $t4, 4($t0)
         bge $t4, $t3, noswp
         sw $t3, 4($t0)
         sw $t4, ($t0)
noswp:   addi $t0, $t0, 4    # v++
         addi $t2, $t2, -1  # j--
         bnez $t2, L1
         addi $t1, $t1, -1  # i--
         la $t0, vect
         addi $t2, $t1, -1  # j = i - 1
         beqz $t2, E1
         j L1
E1:      j E1               # salida

```

El bucle interior está conformado por el loop cuyas líneas de código están entre la etiqueta L1 y la primera instrucción de salto condicional **bnez \$t2, L1**, mientras que el bucle exterior está conformado por las líneas que van desde la misma etiqueta hasta el último salto condicional **beqz \$t2, E1** que permite terminar el programa.

El pseudocódigo de este programa puede ser bastante críptico porque no se pensó nativamente en un lenguaje HLL sino que se partió de una versión optimizada para MIPS.

```

for (i=6; !j; t0 = vect, i--)
    for (j=i-1; !j; t0++, j--) {
        t3 = *t0;
        t4 = *t0+4;
        if (t4 < t3) {
            *t0+4 = t3;
            *t0 = t4;
        }
    }
}

```

1.6. Problemas propuestos de la sección

1. Dado un vector de booleanos `bool` definido como una secuencia de bytes que pueden tomar los valores 0 o 1 inicializado con los siguientes valores `bool=[0,1,1,1,0]` encuentre otro vector de booleanos `res` tal que:

```

res[0] = (bool[0] && bool[4])
res[1] = (bool[1] || bool[3])
res[2] = (bool[0] || bool[1] && bool[2])

```
2. Dato un vector de enteros `V` inicializado con los valores `V=[2,-4,-6]` obtenga un vector de booleanos tal que cada elemento sea 1 si el correspondiente elemento de `V` es mayor o igual que 0.
3. Dato un vector de enteros `V` inicializado con los valores `V=[1,-4,-5,2]` obtenga una variable booleana que será 1 si todos los elementos del vector son menores que 0.
4. Dada una lista de 5 enteros `data1..data5` encuentre si `data5` está en cualquiera de los rangos dado por los pares de los primeros 4 números. Por ejemplo si está entre `data1` y `data2` almacene un 1 en algún lugar reservado llamado `res`.
5. Dado un vector de `n` enteros, contabilice cuantos de ellos son iguales a cero y guarde el resultado en memoria.
6. Diseña un programa en assembly de MIPS que dado un vector de enteros, encuentre el máximo y lo guarde en memoria.
7. Modifique el bubblesort para que ordene la lista en forma descendente.
8. Diseña un programa en assembly de MIPS que detecte si un string cargado en minúsculas y sin acentos es capicua, o sea que si se invierte el orden de sus letras no cambia el mismo por ejemplo `^nana`.